Task-23:- Requirement Analysis

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements, system behavior, and architecture for deploying a **self-hosted GitLab CE environment with distributed GitLab Runners on AWS**.

This document is intended for:

- DevOps Engineers
- System Administrators
- Project Evaluators
- Technical Review Teams

1.2 Scope

Included in Scope:

- Deployment of GitLab CE on AWS EC2
- Setup of two GitLab Runners on separate instances
- Configuration of one runner using AWS Spot Instance
- Creation of a sample project with CI/CD pipeline
- Execution and validation of pipeline

Not Included in Scope:

- High availability or clustered GitLab setup
- Production-grade scaling
- Advanced monitoring and logging setup

1.3 Definitions, Acronyms, and Abbreviations

Term	Description
GitLab CE	GitLab Community Edition (self-hosted Git platform)
CI/CD	Continuous Integration / Continuous Deployment

EC2	Amazon Elastic Compute Cloud
VPC	Virtual Private Cloud
Runner	GitLab agent that executes pipeline jobs
Spot Instance	AWS instance with discounted pricing but interruptible
Pipeline	Automated workflow defined in `.gitlab-ci.yml`

1.4 References

- GitLab Official Documentation: GitLab Docs
- AWS EC2 Documentation: https://docs.aws.amazon.com/ec2
- GitLab Runner Documentation: GitLab Runner | GitLab Docs
- Let's Encrypt (HTTPS): Let's Encrypt

1.5 Overview

This document provides:

- Project overview and objectives
- Functional and non-functional requirements
- Use cases and workflows
- Risk analysis and mitigation strategies

---

2. Project Overview

2.1 Project Objectives

- Deploy GitLab CE on AWS EC2
- Configure two GitLab Runners on separate instances
- Use runner tags for job execution control
- Implement a CI/CD pipeline
- Validate pipeline execution

- Demonstrate cost optimization using Spot Instance

2.2 Business Goals

- Maintain internal control over source code
- Reduce dependency on external CI/CD platforms
- Enable automated software delivery
- Optimize infrastructure cost using Spot Instances

2.3 Background

ABC Technologies requires a **self-hosted DevOps solution** for managing repositories and pipelines internally.

Cloud-based CI/CD tools are costly and less customizable, so the company aims to build a **custom CI/CD infrastructure using GitLab CE on AWS**.

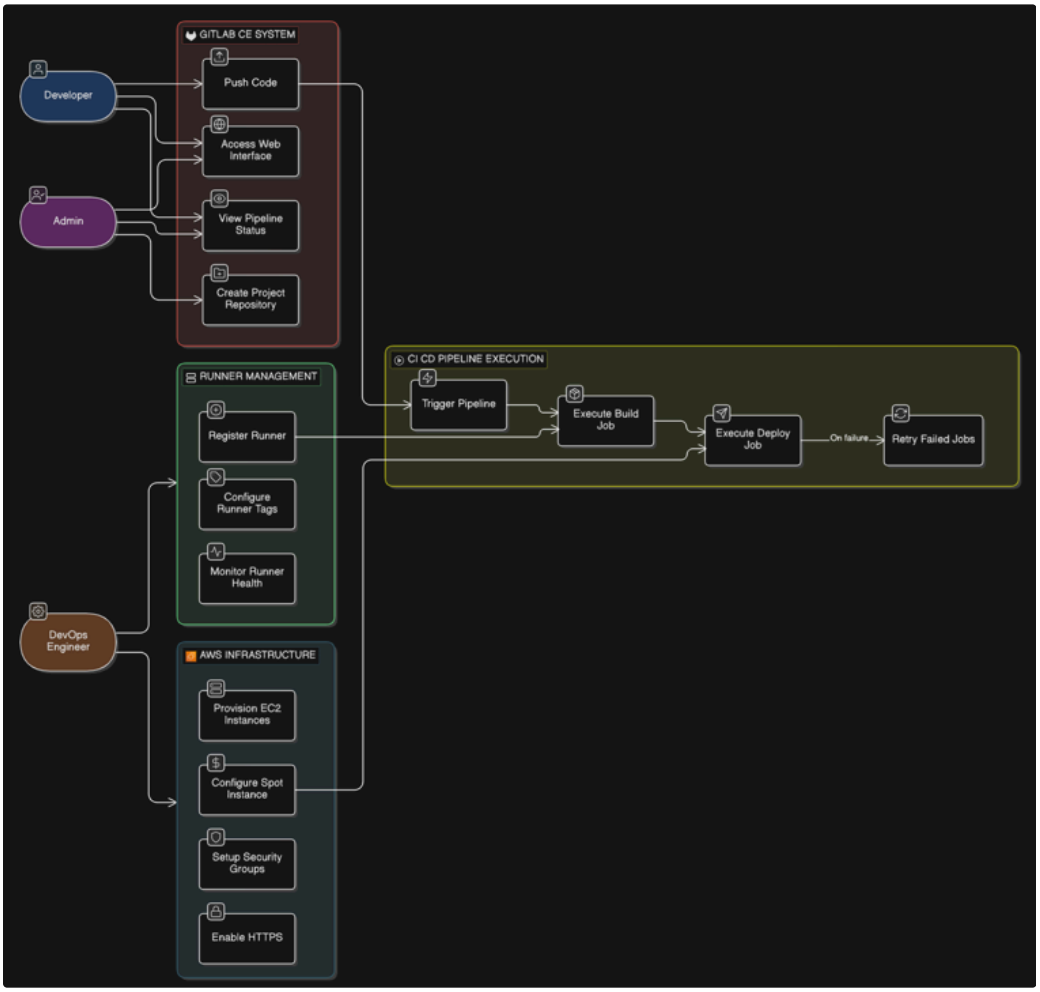
2.4 Constraints

- Time constraint: 3 hours
- Limited AWS resources
- No use of managed GitLab services
- Basic infrastructure only (no HA setup)

---

3. Functional Requirements

3.1 Use Case Diagram



Suggested diagram components:

- Actor: Developer
- System: GitLab CE
- Actions: Push Code, Trigger Pipeline, Execute Jobs

3.2 Use Cases

3.2.1 Use Case 1: Access GitLab Web Interface

Description: User accesses GitLab UI through browser

Actors: Admin / Developer

Preconditions:

- GitLab server is running
- Network access is configured

Postconditions:

- User successfully logs in

Flow of Events:

1. User enters GitLab URL
2. Login page is displayed
3. User enters credentials
4. Dashboard is displayed

Alternative Flows:

- Invalid credentials → Error message shown

3.2.2 Use Case 2: Create Project Repository

Description: Admin creates a new project in GitLab

Actors: Admin

Preconditions:

- User is logged in

Postconditions:

- Repository is created

Flow of Events:

1. Navigate to “New Project”
2. Enter project details
3. Click Create
4. Repository is created

Alternative Flows:

- Missing fields → Validation error

3.2.3 Use Case 3: Register GitLab Runner

Description: Register runner with GitLab server

Actors: DevOps Engineer

Preconditions:

- Runner installed on EC2
- Registration token available

Postconditions:

- Runner appears online in GitLab

Flow of Events:

1. Install GitLab Runner
2. Execute registration command
3. Provide GitLab URL and token
4. Assign tags
5. Runner registered successfully

Alternative Flows:

- Invalid token → Registration fails

3.2.4 Use Case 4: Execute CI/CD Pipeline

Description: Pipeline runs on code push

Actors: Developer

Preconditions:

- `.gitlab-ci.yml` exists
- Runners are active

Postconditions:

- Pipeline executed successfully

Flow of Events:

1. Developer pushes code
2. GitLab triggers pipeline
3. Jobs assigned based on tags
4. Build job runs on Runner 1
5. Deploy job runs on Runner 2

6. Pipeline completes

Alternative Flows:

- Runner offline → Job pending
- Job failure → Pipeline fails

3.2.5 Use Case 5: Execute Job on Spot Instance Runner

Description: Deploy job runs on Spot Instance

Actors: GitLab System

Preconditions:

- Spot runner registered
- Runner is available

Postconditions:

- Job executed successfully

Flow of Events:

1. Pipeline triggers deploy stage
2. GitLab assigns job to Spot runner
3. Runner executes job
4. Status updated

Alternative Flows:

- Spot instance terminated → Job retried

---

4. Non-Functional Requirements

4.1 Performance Requirements

- Pipeline execution should complete within reasonable time
- Runners should respond quickly to job assignments

4.2 Security Requirements

- HTTPS must be enabled
- Secure SSH access via key pairs

- Restricted ports using security groups

4.3 Usability Requirements

- Simple and intuitive GitLab UI
- Easy runner registration process
- Clear pipeline status visibility

4.4 Reliability Requirements

- GitLab uptime should be stable
- Pipeline jobs should retry on failure
- Runner availability must be monitored

4.5 Compliance Requirements

- Follow AWS security best practices
- Ensure secure handling of credentials

---

5. Assumptions and Dependencies

5.1 Assumptions

- AWS account is available
- Internet connectivity exists
- Basic Linux knowledge is available

5.2 Dependencies

- AWS EC2 service
- GitLab installation packages
- Network configuration (VPC, Security Groups)

---

6. Risk Analysis

6.1 Risk Identification

Risk	Description
Spot Instance interruption	Runner may stop unexpectedly
Network issues	Runners may go offline
Misconfiguration	Pipeline may fail
HTTPS setup failure	Security risk

6.2 Risk Mitigation

Risk	Mitigation
Spot interruption	Retry jobs / use backup runner
Network issues	Proper security group configuration
Misconfiguration	Validate setup step-by-step
HTTPS failure	Use Let's Encrypt properly

---

7. Glossary

- **Pipeline:** Automated CI/CD workflow
- **Runner:** Agent executing pipeline jobs
- **Repository:** Code storage location
- **Stage:** Logical phase in pipeline (build, deploy)

---

8. Appendices

- GitLab Dashboard
- Project Repository
- Runner List
- Pipeline Execution
- Spot Instance Proof

---

9. Customer Acknowledgement

Name: _____

Title: _____

Signature: _____

Date: _____